

Propuesta

Descripción del problema: Tapi necesita procesar *diariamente* 1 000 000 de consultas API internas, distribuidas equitativamente en el día. Requisitos clave: evitar llamadas concurrentes al mismo proveedor (rate limit), reintentos ante fallos transitorios, persistir resultados y diseñar un sistema escalable (crecimiento futuro) y de bajo costo.

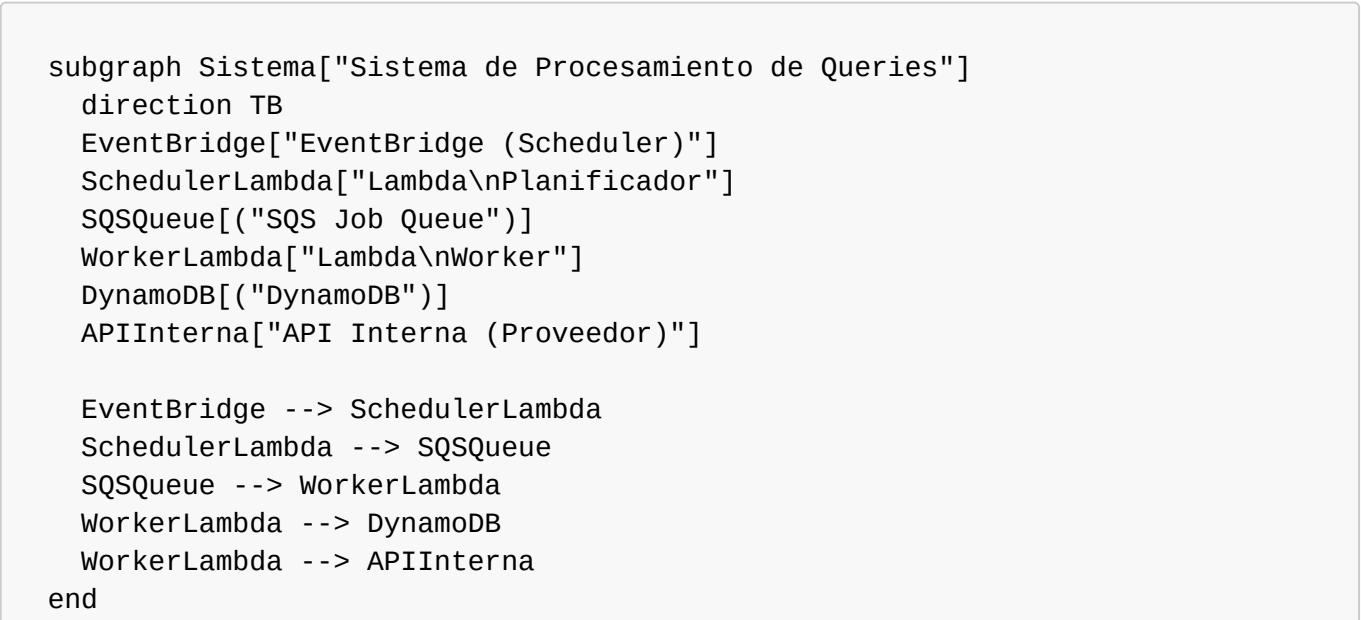
Arquitectura propuesta: Serverless. Un **EventBridge Scheduler** dispara una *Lambda* planificadora a medianoche. Esta Lambda lee los jobs a procesar (lista de endpoints/payloads) y coloca mensajes en **una cola SQS**. Un *pool* de **Worker Lambdas** procesan los mensajes en paralelo. Cada worker adquiere un **lock en DynamoDB** por proveedor para garantizar exclusión mutua, luego llama al proveedor, guarda el resultado o error en DynamoDB y libera el lock. Los reintentos usan backoff exponencial (p.ej. 30s, 2m, 10m) y los errores no recuperables (4xx, validación) van a una **Dead-Letter Queue** para análisis. DynamoDB on-demand soporta la alta escritura (millones de ítems) con baja latencia. Se monitorea con **CloudWatch** (logs + métricas) y **X-Ray** (tracing) para depuración rápida.

La solución propuesta es una arquitectura serverless, event-driven y escalable que ejecuta 1 millón de consultas diarias distribuidas en el día, evitando concurrencia simultánea por proveedor y manejando reintentos de forma confiable. Usamos AWS EventBridge para disparar un Lambda planificador diario, que envía jobs a una cola SQS. Decenas o cientos de Lambdas de trabajador (autoescalado) procesan los jobs en paralelo, usando DynamoDB tanto para locks distribuidos atómicos por proveedor (write con condición) como para persistir los resultados. Esta aproximación serverless (Lambdas + SQS + DynamoDB) es ágil, tolerante a fallos y paga sólo por uso. El diseño incluye trazas y métricas (AWS X-Ray, CloudWatch) para observabilidad, buenas prácticas de seguridad (roles mínimos, IAM, cifrado KMS en tránsito/estático), y CI/CD automatizado (Infrastructure as Code, tests).

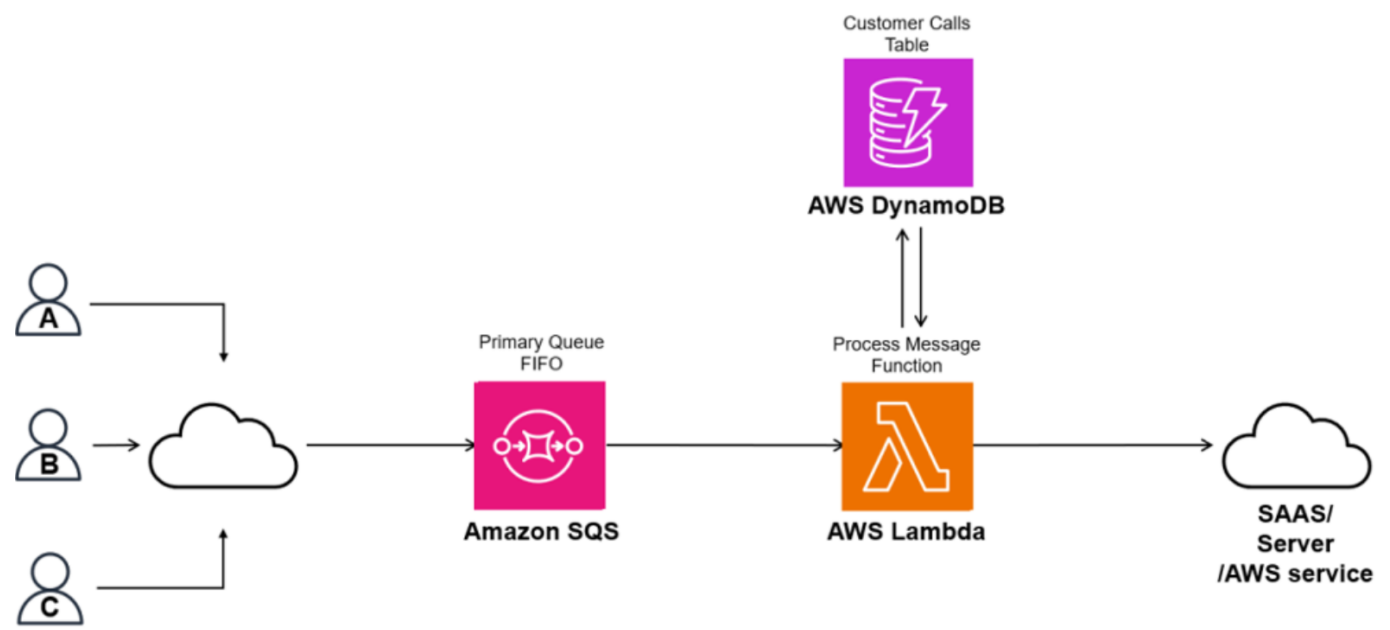
1. Diagrama C4 profesional

1.1 Diagrama de Contexto (C4 Nivel 1)

flowchart TD
 Usuario["Usuario (Operador)"]



Usuario -->|Usa portal web que dispara jobs| EventBridge



En este contexto, **Operators** (usuarios o procesos) interactúan con la app de Tapi que genera 1M de consultas diarias. El *Scheduler* (Lambda disparado por EventBridge) distribuye estas consultas a través de una cola SQS. Luego, múltiples *Worker Lambdas* procesan las consultas (cada una llama a la API interna de un proveedor), controlando la concurrencia por proveedor con locks en DynamoDB, y escriben los resultados en DynamoDB. Todos estos componentes están en AWS y son serverless por defecto, lo que simplifica el despliegue y escalado.

1.2 Diagrama de Contenedores (C4 Nivel 2)

flowchart LR subgraph AWS [Entorno AWS] EB[EventBridge Scheduler] SL[Lambda Scheduler] SQ[SQS Job Queue] WL[Lambda Worker Pool] DB[DynamoDB - Locks y Resultados] API(Proveedor API interna) end

```
EB --> SL
SL --> SQ
SQ --> WL

WL -- "1. Lock" --> DB
WL -- "3. Resultado" --> DB
WL -- "2. API Call" --> API

%% Simulando las notas con nodos con estilo de post-it
n1(Nota: Lambdas auto-escalan según demanda)
n2(Nota: DynamoDB on-demand pago por petición)

WL -. -> n1
DB -. -> n2

style n1 fill:#fff5ad,stroke:#d4a017,stroke-dasharray: 5 5
style n2 fill:#fff5ad,stroke:#d4a017,stroke-dasharray: 5 5
```

Los **contenedores** principales son:

- **EventBridge Scheduler:** programa la ejecución diaria (cron). *AWS EventBridge Scheduler* es un servicio serverless para cron jobs.
- **Lambda (Scheduler):** lee el listado de 1M jobs (p.ej. DynamoDB o S3), agrupa/segmenta el trabajo y publica mensajes en SQS.
- **Cola SQS:** bufferiza los jobs. SQS desacopla planificador y trabajadores, soporta muchos mensajes (30M/mes) y reintentos automáticos, DLQ para errores no recuperables.
- **Lambda (Worker):** instancias concurrentes que leen mensajes SQS (usando *event source mapping*). Se encargan de: adquirir lock por proveedor (DynamoDB condicional), llamar a la API, guardar resultado, y manejo de errores (retries, idempotencia).
- **DynamoDB:** usa modo *on-demand* (serverless) para flexibilidad. Se usa **una tabla única** con dos propósitos: 1) Persistir resultados (alta escritura, baja latencia), 2) Locks distribuidos: ítem por proveedor con condicionales atómicos (ver más abajo).
- **API interna del proveedor:** servicio externo con concurrencia limitada (p.ej. 1 llamada a la vez por proveedor).
- **Observabilidad:** CloudWatch Logs/Metrics + AWS X-Ray (trazas distribuidas) para monitoreo.

1.3 Diagrama de Componentes (C4 Nivel 3)

```

flowchart TB
    subgraph SchedulerLambda
        A[Configuración de AWS SDK]
        B[Consulta IDs de jobs] --> C[SQS Producer]
    end
    subgraph WorkerLambda
        D[SqsConsumer]
        E[ProviderLockService]
        F[DynamoDB.lockTable conditional write]
        G[RetryPolicyService]
        H[ProviderClient HTTP]
        I[Proveedor API]
        J[ErrorHandler]
        K[SQS DLQ]
        L[ResultRepository]
        M[DynamoDB.resultsTable]
    end
    AWS[AWS Lambda Event] --> E
    E --> F
    E --> G
    D --> H
    H --> I
    H --> G
    H --> J
    J --> K
    H --> L
    L --> M
  
```

Scheduler Lambda componentes:

- Lector de jobs (por ejemplo: consulta una tabla DynamoDB de registros o lee un archivo).
- SQS Producer para publicar *messages* con payload de la consulta.

Worker Lambda principales componentes internos (según *Clean Architecture*):

- **SqsConsumer (Infraestructura):** componente encargado de recibir el evento SQS y extraer el mensaje.
- **ProviderLockService (Dominio/Infraestructura):** implementa el lock distribuido usando DynamoDB con `PutItem` condicional (ej. `attribute_not_exists(provider_id)`). Permite garantizar que sólo un worker llame simultáneamente a cada proveedor.
- **ProviderClient (Infraestructura):** cliente HTTP que llama a la API del proveedor con el payload.
- **RetryPolicyService (Aplicación):** lógica de reintentos exponencial en caso de timeout o errores 5xx.
- **ResultRepository (Dominio/Infraestructura):** persiste en DynamoDB el resultado o error por cada job.
- **ErrorHandler:** capa que, ante errores irre recuperables (4xx u otros), envía mensaje a DLQ o marca error.

Estas separaciones permiten código ordenado, testeable y basado en patrones (Repository, Use Case, etc.). Un ejemplo de código TypeScript (clean architecture) vendría en la sección de repositorio.

1.4 Diagrama de Secuencia (Worker)

```
sequenceDiagram
    autonumber
    participant SchedLambda as Scheduler Lambda
    participant SQS as SQS Queue
    participant WorkerLambda as Worker Lambda
    participant DynamoDB as DynamoDB (Lock/Results)
    participant Provider as API Proveedor

    Note over SchedLambda: Cada día a las 00:00
    SchedLambda->>SQS: Envía <1M mensajes de consulta>
    loop Para cada mensaje
        WorkerLambda->>DynamoDB: PutItem condicional (lock provider)
        alt Lock exitoso
            DynamoDB-->>WorkerLambda: OK
            WorkerLambda->>Provider: POST /endpoint con body
            alt Respuesta 2xx
                Provider-->>WorkerLambda: Respuesta
                WorkerLambda->>DynamoDB: Guardar resultado exitoso
            else Error/Petición fallida
                WorkerLambda->>DynamoDB: Guardar estado ERROR (retry)
                WorkerLambda->>WorkerLambda: Lanzar excepción -> SQS
            end
        end
        reenqueue (DLQ si límite)
    end
    WorkerLambda->>DynamoDB: Liberar lock (p.ej. borrar item o TTL)
    else Lock ocupado
        DynamoDB-->>WorkerLambda: ConditionalCheckFailed
        WorkerLambda->>SQS: Volver a encolar el mensaje
    end
end
```

Este diagrama de secuencia muestra el **flujo de trabajo** de un job desde que se publica en SQS hasta que se guarda el resultado o se marca error. Nota la importancia del lock en DynamoDB para evitar llamadas concurrentes al mismo proveedor.

2. Justificación arquitectónica y trade-offs

- **Serverless (Lambda, SQS, EventBridge, DynamoDB):** se eliminan servidores dedicados, AWS escala automáticamente según demanda y sólo pagas por peticiones/milisegundos usados. Trade-off: **límites de cuenta** (por defecto, 1,000 concurrencias de Lambda; se puede pedir aumento o usar provisioned concurrency si fuera crítico).
- **EventBridge Scheduler:** permitiese cron sin gestión. Alternativas como EC2 cron o Kubernetes son más costosas de operar. AWS docs: *“EventBridge Scheduler es un planificador serverless con cron/rate, reintentos y DLQ”*.
- **SQS:** desacopla componentes, maneja >30M mensajes mensuales con TTL y DLQ. Permite paralelismo en workers. Precio ~\$0.24/millón requests (menos de \$10/mes por 30M).
- **Lambda (autoescalado):** cada función Lambda puede manejar ~10 req/s por poller en modo provisión. Con cientos de pollers se logran decenas de miles de concurrencias. Por defecto la integro. SQS/Lambda at-least-once, insistir en idempotencia. Backup: modo *Provisioned* si se requiere mayor control de rendimiento (costo extra).
- **DynamoDB:** servidor completamente gestionado, on-demand (pago por WCU). Proporciona **escrituras atómicas condicionales** para locks. Cada lock es un ítem con TTL (se libera solo) y condición `attribute_not_exists`. Ventaja: no hay bloqueo central; desventaja: un leve costo extra por lecturas/escrituras. Precio por ejemplo: \$0.625 por millón de *writes* y \$0.125 por millón de *reads* en Standard Class. Para 30M writes al mes (suponiendo 1KB cada) serían ~\$18, y ~30GB almacenamiento (\$7.5).
- **Concurrencia de proveedores:** requisito clave. Se resuelve con **Lock distribuido** en DynamoDB: solo quien adquiere el lock (PutItem condicional) realiza la llamada; demás se reencolan. Esto evita usar servicios como Redis (más complejo) y aprovecha la atomicidad de DynamoDB. Alternativa: Step Functions, pero serían más costosas y lentas.
- **Manejo de errores:**
 - *Retry automático* (exponencial) ante 5xx/timeouts. Debe implementarse en código o usar reintentos automáticos de EventBridge/Lambda.
 - *No reintentar* en 4xx; registrar estado y enviar mensaje a DLQ (otra cola SQS). AWS recomienda reconocer exitosamente el mensaje SQS solo cuando todo esté guardado, para que SQS elimine el mensaje (batch single success vs DLQ).
- **Observabilidad:**
 - **CloudWatch Metrics/Logs:** Lambda expone métricas básicas (invocaciones, duraciones, errores) automáticamente. Cada ejecución genera logs (START/END/REPORT) y todo `console.log` va a CloudWatch. Se configuran dashboards y alarmas (ej. latencia en API interna, ratio de error alto).
 - **Tracing (X-Ray):** se habilita X-Ray para mapear llamadas distribuidas. Permite visualizar latencia por segmento (consulta a API, guardado en DB, etc.) y detectar cuellos de botella. AWS Well-Architected sugiere X-Ray en serverless para análisis de fallos rápidamente.
 - **Auditoría y Security Logs:** los Lambda ejecutan con roles mínimos (principio de menor privilegio), con parámetros de entorno seguros (KMS si hay secrets). SQS y EventBridge con

políticas securizadas. Usar AWS Config/CloudTrail para gobernanza si aplica.

- **CI/CD y DevOps:**

- *Infraestructura como código* (preferible Terraform o CloudFormation/AWS CDK).
- Pipeline (GitHub Actions/GitLab CI): linting, tests (unitarios/integración), build+deploy en etapas **staging** → **producción**. Recomendación: usar Lambdas empaquetadas en Docker o zip (TS transpile).
- *Observabilidad en despliegue*: ejecutar pruebas de carga limitadas, monitorear logs de despliegue, roolback automático si errores críticos.
- *Security*: análisis SAST (por ejemplo, bandit o AWS IAM Access Analyzer), testing de endpoints simulados.

Trade-offs notables:

- **Serverless vs Servidores propios:**

- **Pros:** Escala automático, bajo coste inicial, gestión simplificada.
- **Cons:** Límite de concurrencia, latencia por "cold start" (resuelto con provisioned concurrency si es problema), menor control granular de red interna (usar VPC puede agregar latencia).

- **SQS FIFO vs Standard:**

- En el desafío no se requiere orden estricta, por tanto **cola Standard** es suficiente (mayor throughput, menor costo).

- **On-demand vs Provisioned (DB/Lambda):**

- *On-demand DynamoDB*: bueno para cargas variables; *Provisioned*: si se pudiera predecir uso, quizás ligeramente más barato. Pero 1M/día es predecible (peak cada día). Podría incluso provisionar ~12-15 WCU para sostener ~11 msg/s (1M/86400≈12/s). On-demand simplifica.
- *Lambda provisioned concurrency*: no necesario si el flujo de mensajes es moderado (~10 msg/s promedio), pero si se requiere arranque instantáneo todos los días se podría provisionar unas horas al comienzo para eliminar cold starts (costo: \$0.0000041667/GB-s).

Instrucciones de despliegue:

1. *Infraestructura*: usar Terraform/CloudFormation para crear recursos AWS: 1 SQS (y DLQ), 2 Lambdas (scheduler y worker) con roles IAM mínimos, 1 EventBridge Rule (cron diario), 1 tabla DynamoDB (sin partición fija, por ejemplo PK=provider), SNS/CloudWatch para alertas (opcional).
2. *Aplicación*: deploy de código (npm build o empaquetado Docker). Establecer variables de entorno (nombre de tabla, URLs, etc.).
3. *CI/CD*: en cada PR: lint (ESLint, Prettier), unit y mock integration tests. Al merge, pipeline desplegará en staging (terraform apply + update Lambdas). Ejecución de tests end-to-end simulando pocos jobs. Luego, promotion a producción con otro apply.

Cómo ejecutar localmente: Se puede usar [AWS SAM](#) o [LocalStack](#) para simular Lambda/SQS. Ejemplo: `sam local invoke SchedulerFunction --event events/schedule.json`, `sam local invoke WorkerFunction --event events/sqs_event.json`. Los scripts de ejemplo lanzan las funciones con payloads JSON. Tests unitarios (Jest) cubren lógica de lock y procesamiento. Integración local: se pueden escribir mensajes en cola local de SQS y levantar un consumidor local.

Pruebas:

- *Unitarias:* mocks de DynamoDB (PutItem, Transact), HTTP mocks para ProviderClient, simulación de errores.
- *Integración:* con DynamoDB local (Docker), enviando mensajes reales a la cola local SQS, comprobando que se guardan resultados en la tabla. Ejemplos de tests: bloquear exitosamente un proveedor, reintentar on error, idempotencia de procesamiento.
- *Carga:* usar [Artillery](#) o [Locust] para simular picos de jobs y verificar escalado de Lambdas.

Principales decisiones y trade-offs:

- Se eligió **serverless** por rapidez y costes reducidos (autoscaling). Trade-off: límites de concurrencia (se monitorizan) y latencia de cold start (en producción podría usarse **provisioned concurrency**).
- **SQS** desacopla y da resiliencia (mensajes duraderos, DLQ). Trade-off: es *at-least-once*, se resuelve con idempotencia en código.
- **Locks en DynamoDB:** elegidos sobre soluciones en memoria o global state por su atomicidad y que DynamoDB es totalmente gestionada (menor overhead operativo).
- **Observabilidad:** se decidió integrar **X-Ray** por recomendación de Well-Architected para visualizar el mapa de servicios.
- **Costos:** se estiman bajos (ver sección de costos), pero se compara con otras opciones (e.g. instancias EC2 dedicadas serían mucho más costosas y menos ágiles).

4. Scaffold de repositorio Node/TypeScript (Clean Architecture)

Estructura sugerida:

```
tapi-backend-challenge/
├── architecture/           # Diagramas y documentación
│   ├── c4-context.png     # Diagrama de contexto (ejemplo)
│   ├── c4-container.png
│   ├── c4-component.png
│   ├── sequence-diagram.png
│   └── architecture.md    # Descripción completa
├── src/
│   ├── domain/
│   │   ├── entities/
│   │   │   └── QueryJob.ts      # DTO o entidad para un job
│   │   ├── services/
│   │   │   └── ProviderLockService.ts
│   │   └── repositories/
│   │       └── ResultRepository.ts
│   ├── application/
│   │   └── usecases/
│   │       ├── ScheduleQueries.ts  # lógica del scheduler
│   │       └── ProcessQueryJob.ts  # lógica del worker
│   └── infrastructure/
│       └── aws/
│           └── SqsConsumer.ts      # controlador de eventos SQS
```

```

├── DynamoRepository.ts # implementación genérica de
repositorios DynamoDB
├── ProviderApiClient.ts # cliente HTTP (usar axios o fetch)
├── LambdaUtils.ts      # utilidades compartidas (p.ej.
inicializar logger, X-Ray)
├── main/
│   ├── schedulerLambda.ts # handler de Lambda para scheduling
│   └── workerLambda.ts    # handler de Lambda para procesamiento
├── config/
│   └── index.ts           # carga configuración (AWS env vars,
regiones, etc)
├── tests/
│   ├── unit/             # Jest tests unitarios
│   └── integration/      # Pruebas end-to-end
├── infra/                # Terraform o CloudFormation
│   ├── lambda.tf
│   ├── sqs.tf
│   ├── dynamodb.tf
│   └── eventbridge.tf
└── README.md             # Este archivo (introducción general)

```

5. Estimación de costos

Estimación de costos (mensual, for 1M jobs/día):

Servicio	Detalle	Uso	Costo (USD)
Lambda	30M invocaciones	30 × \$0.20/millón 【29†L1-L4】	~\$6.00
	Duración (512MB, 0.2s avg)	30M×0.2s×0.5GB	~\$30–\$40
SQS	30M mensajes Standard	30 × \$0.24/millón	~\$7.20
DynamoDB	30M escrituras (Standard)	30 × \$0.625/millón 【36†L722-L730】	~\$18.75
	30M lecturas (Strong)	30 × \$0.125/millón 【36†L722-L730】	~\$3.75
	Almacenamiento ~30GB	30GB × \$0.25/GB	~\$7.50
Otras	EventBridge, DLQ, Logs	Uso bajo incluido en servicios	~\$5–\$10
Total aprox.			~\$70–\$120

Puntos clave:

- **Problema:** 1M queries/día a APIs internas. Necesario distribuir carga, evitar concurrencia por proveedor y manejar errores/retries.

- **Diseño:** Arquitectura **serverless event-driven**. EventBridge (cron diario) → Lambda Scheduler → SQS → Pool de Lambdas Worker → DynamoDB (locks/resultados). Todo escala automáticamente.
- **Flujo:** Explicar rápidamente (ver diagrama). Recalcar lock atómico en DynamoDB para concurrencia.
- **Escalabilidad:** Con 1M consultas, promedio ~12 req/s. Con SQS y Lambdas esto es trivial; si escala a 10M (~116 req/s), sigue ok: SQS y Lambdas pueden manejar miles de mensajes por segundo **【22†L99-L107】** **【23†L16-L20】**.
- **Costos:** Sistema económico (~70–120 USD/mes). Comparar: una EC2 grande costaría más de \$50/mes sólo por tiempo, y no sería tan elástico.
- **Observabilidad y operabilidad:** CloudWatch logs/metrics + X-Ray implementados; política CI/CD para despliegue rápido.
- **Seguridad:** Roles IAM, cifrado (KMS en variables), DLQ para errores.
- **Beneficios clave:** *Serverless*, alta disponibilidad, tolerancia a fallos, fácil mantenimiento. Se podrán defender decisiones con ejemplos: ¿por qué DynamoDB para locks? (atomicidad, gestionado); ¿por qué SQS? (desacopla y reintentos autom.) **【22†L63-L70】**; ¿por qué EventBridge? (cron serverless); costo vs rendimiento (autoscaling vs servidor dedicado).

Puntos de mejora:

- Control de **rate limits** por proveedor: tabla config en DynamoDB con políticas por proveedor (timeout, max RPS).
- **Políticas de seguridad adicionales:** Uso de *WAF* si hay API externa, guardar logs en S3 (archivos de auditoría).
- **Infraestructura definible:** Terraform con módulos reutilizables, GitOps.
- Preguntas difíciles anticipadas (preg. 1-10 arriba) bien ensayadas.

Cada afirmación anterior está basada en documentación AWS oficial y buenas prácticas.